Lecture Notes: TwinCAT 3

Function Blocks and Object-Oriented Programming (OOP)

March 14, 2026

Contents

1 Procedural vs. Object-Oriented Programming **2**

2 Function Blocks as Blueprints **2**

3 The Pillars of OOP in TwinCAT 3 **3**

3.1 Encapsulation 3

3.2 Abstraction 3

3.3 Inheritance 3

4 Methods and Access Specifiers **4**

5 FB Body vs. Methods **5**

6 Best Practices for Professional Code **5**

6.1 Self-Descriptive Code 5

6.2 Avoid "Magic Numbers" 5

6.3 Modularization 5

7 Practical Example: The Event Logger **6**

Quick Reference Card **8**

1 Procedural vs. Object-Oriented Programming

The transition from functions to **function blocks (FBs)** represents a shift from **Procedural Programming (POP)** to **Object-Oriented Programming (OOP)**.

- **Procedural (POP):** Focuses on a sequence of actions. Global data is defined, and functions act upon that data.
- **Object-Oriented (OOP):** Focuses on data. Programs are divided into **objects** that carry their own state (data) and behavior (methods).
- **The Object:** In industrial automation, an object can represent physical hardware (a motor, a sensor) or abstract concepts (a database logger, a mathematical calculation).

Aspect	Procedural (POP)	Object-Oriented (OOP)
Focus	Sequence of actions	Data and objects
Data Location	Global variables	Encapsulated in objects
Code Reuse	Functions	Classes/Function Blocks
State Management	External to functions	Internal to objects

Table 1: Comparison of Programming Paradigms

2 Function Blocks as Blueprints

A function block serves as the **blueprint** for an object.

- **State:** Defined by internal variables (e.g., a motor's current velocity or temperature).
- **Behavior:** Defined by **methods** (e.g., `PowerEnable`, `SetSpeed`).
- **Instantiation:** To use an FB, you must create an **instance** of it. This reserves memory in the PLC, allowing multiple objects (e.g., `fbWinch` and `fbConveyor`) to exist independently, each holding its own unique data.

Example: Instantiation

```
// Program: MAIN
// Declaring multiple instances of the Motor function block
// Each instance maintains its own independent state and data
PROGRAM MAIN
VAR
    fbWinch    : FB_Motor;    // First instance - controls the winch motor
    fbConveyor : FB_Motor;    // Second instance - controls the conveyor motor
    fbPump     : FB_Motor;    // Third instance - controls the pump motor
END_VAR

// Each instance can be controlled independently
fbWinch(bEnable := TRUE, fTargetSpeed := 50.0);
fbConveyor(bEnable := TRUE, fTargetSpeed := 75.0);
fbPump(bEnable := FALSE, fTargetSpeed := 0.0);
```

```

// Access instance-specific data
// fbWinch.fCurrentSpeed will be different from fbConveyor.fCurrentSpeed
```

Note: Each instance of a function block has its own memory allocation. This means `fbWinch` and `fbConveyor` can have completely different internal states even though they are created from the same blueprint.

3 The Pillars of OOP in TwinCAT 3

3.1 Encapsulation

Bundling data with the methods that operate on it and hiding the internal state from the outside. This prevents "spaghetti code" caused by excessive global variables.

3.2 Abstraction

Hiding complex internal logic from the user. The caller only needs to know *what* a method does (e.g., `AddEvent`), not the specific details of *how* it is accomplished.

3.3 Inheritance

The process where a **child function block** inherits all attributes and methods of a **parent function block**.

- The child can add its own specific features or methods.
- **"IS-A" Relationship:** An SMS notification *is a* notification.
- **Syntax:** Use the `EXTENDS` keyword to implement inheritance.

Example: Inheritance

```
// Parent Function Block: FB_Notification
// Base class containing common notification functionality
FUNCTION_BLOCK FB_Notification
VAR
    sRecipient    : STRING(255);     // Who receives the notification
    sMessage      : STRING(1000);    // The notification content
    dtTimestamp   : DATE_AND_TIME;   // When the notification was created
    bSent         : BOOL;            // Whether notification has been sent
END_VAR

// Methods for the parent FB would be defined here
// e.g., Send(), GetStatus(), etc.
```

```
// Child Function Block: FB_SMSNotification
// Inherits from FB_Notification and adds SMS-specific functionality
FUNCTION_BLOCK FB_SMSNotification EXTENDS FB_Notification
VAR
```

```
    sPhoneNumber     : STRING(20);      // SMS-specific: recipient phone number
    nMaxCharacters   : INT := 160;      // SMS-specific: character limit
    bDeliveryReport  : BOOL;            // SMS-specific: request delivery confirmation
END_VAR

// This FB automatically has access to:
// - sRecipient (inherited)
// - sMessage (inherited)
// - dtTimestamp (inherited)
// - bSent (inherited)
// Plus its own SMS-specific variables
```

```
// Another Child: FB_EmailNotification
// Different specialization of the base notification class
FUNCTION_BLOCK FB_EmailNotification EXTENDS FB_Notification
VAR
    sEmailAddress : STRING(255);      // Email-specific: recipient address
    sSubject      : STRING(255);      // Email-specific: email subject line
    sAttachment   : STRING(255);      // Email-specific: attachment path
    bHTMLFormat   : BOOL;             // Email-specific: HTML or plain text
END_VAR
```

Tip: Use inheritance when you have a clear "IS-A" relationship. For example, `FB_SMSNotification` IS-A `FB_Notification`. This promotes code reuse and maintains a logical hierarchy.

4 Methods and Access Specifiers

Methods are used to change or access the internal state of an object. Every method requires an **access specifier** to define who can call it:

Specifier	Symbol	Accessibility
`PUBLIC`	+	Accessible by everyone (the FB itself and external code)
`PRIVATE`	-	Accessible only within the FB where it is declared
`PROTECTED`	#	Accessible by the FB and its children (via inheritance)
`INTERNAL`	~	Accessible within the FB and its local library

Table 2: Access Specifiers in TwinCAT 3

Access From	PUBLIC	PRIVATE	PROTECTED	INTERNAL
Same FB	✓	✓	✓	✓
Child FB	✓		✓	✓
Same Library	✓			✓
External Code	✓			

Figure 1: Access Specifier Visibility Matrix

5 FB Body vs. Methods

Unlike traditional languages like Java where all state changes must occur through methods, TwinCAT allows you to call the **body** of the function block directly.

- **Cyclical Logic (The Body):** Use the FB body for code that must run every single PLC cycle.
- **One-Shot Logic (Methods):** Use methods for specific requests or messages to change state that do not need to run every cycle.

Aspect	FB Body	Methods
Execution	Every PLC cycle	On-demand (when called)
Use Case	Continuous monitoring/control	Specific actions/requests
Example	Motor speed regulation loop	`SetTargetSpeed()`

Table 3: FB Body vs. Methods

6 Best Practices for Professional Code

6.1 Self-Descriptive Code

Prioritize descriptive variable and method names over comments. Comments often become outdated as code changes, leading to confusion.

6.2 Avoid "Magic Numbers"

Use **constants** for fixed values (like buffer sizes) to make the code more readable.

6.3 Modularization

Create small function blocks that do "one thing only" but do it very well.

Example: Constants vs. Magic Numbers

```
// BAD PRACTICE: Using "Magic Numbers"
// What does 100 mean? Why 100? Hard to understand and maintain.
FUNCTION_BLOCK FB_EventLogger_Bad
VAR
    astEventBuffer : ARRAY[0..99] OF ST_Event;  // Why 99? What if we
    need to change it?
    nCurrentIndex  : UDINT;
END_VAR
```

```

// Checking against magic number - unclear intent
IF nCurrentIndex < 100 THEN
    // Add event...
END_IF
```

```
// GOOD PRACTICE: Using Named Constants
// Clear, self-documenting, easy to maintain
FUNCTION_BLOCK FB_EventLogger_Good
VAR CONSTANT
    MAXIMUM_BUFFER_SIZE : UDINT := 100;     // Single point of definition
    MINIMUM_EVENT_ID    : UDINT := 1;        // Descriptive name explains purpose
    INVALID_INDEX       : UDINT := 16#FFFFFFFF;  // Clear meaning
END_VAR
VAR
    // Array size defined using the constant - change in one place updates everywhere
    astEventBuffer : ARRAY[0..(MAXIMUM_BUFFER_SIZE - 1)] OF ST_Event;
    nCurrentIndex  : UDINT := 0;
    nEventCount    : UDINT := 0;
END_VAR

// Clear intent: checking against named constant
IF nCurrentIndex < MAXIMUM_BUFFER_SIZE THEN
    // Add event...
END_IF
```

Warning: Magic numbers make code difficult to maintain. If you need to change a buffer size from 100 to 200, you would have to find and update every occurrence of "100" in your code. With constants, you change it in one place.

7 Practical Example: The Event Logger

The tutorial demonstrates building an `EventLogger` function block to store a fixed-size buffer of events.

- **Internal Buffer:** An array of event structures.
- **Public Method (`AddEvent`):** Allows external code to submit an event identity, text, etc.
- **Private Method (`IsEventBufferFull`):** An internal helper method used by `AddEvent` to check if the buffer has reached its maximum capacity before adding new data.

Example: Private Helper Method

```
// Function Block: FB_EventLogger
// A circular buffer for storing system events
FUNCTION_BLOCK FB_EventLogger
VAR CONSTANT
    MAXIMUM_BUFFER_SIZE : UDINT := 100;     // Maximum events to store
END_VAR
VAR
```

```
    astEventBuffer : ARRAY[0..(MAXIMUM_BUFFER_SIZE - 1)] OF ST_Event;
    nCurrentIndex  : UDINT := 0;            // Current write position
    nEventCount    : UDINT := 0;            // Total events stored
    bBufferFull    : BOOL := FALSE;         // Buffer status flag
END_VAR
```

```
// Private Method: IsEventBufferFull
// Checks if the event buffer has reached maximum capacity
// Access: PRIVATE - can only be called from within FB_EventLogger
METHOD PRIVATE IsEventBufferFull : BOOL
VAR_INPUT
    // No inputs needed - uses internal FB variables
END_VAR

// Compare current index against the maximum buffer size constant
// Returns TRUE if buffer is full, FALSE otherwise
IF nCurrentIndex >= MAXIMUM_BUFFER_SIZE THEN
    IsEventBufferFull := TRUE;
ELSE
    IsEventBufferFull := FALSE;
END_IF

// Alternative one-liner:
// IsEventBufferFull := (nCurrentIndex >= MAXIMUM_BUFFER_SIZE);
```

```
// Public Method: AddEvent
// Adds a new event to the buffer if space is available
// Access: PUBLIC - can be called from external code
METHOD PUBLIC AddEvent : BOOL
VAR_INPUT
    nEventId     : UDINT;           // Unique event identifier
    sEventText   : STRING(255);     // Event description
    eSeverity    : TcEventSeverity; // Event severity level
END_VAR
VAR
    fbGetSystemTime : GETSYSTEMTIME;
    stFileTime      : T_FILETIME;
END_VAR

// First check if buffer is full using our private helper method
IF THIS^.IsEventBufferFull() THEN
    // Buffer is full - cannot add new event
    AddEvent := FALSE;
    RETURN;
END_IF

// Buffer has space - add the new event
astEventBuffer[nCurrentIndex].nIdentity      := nEventId;
astEventBuffer[nCurrentIndex].sText          := sEventText;
astEventBuffer[nCurrentIndex].eEventSeverity := eSeverity;

// Get current timestamp
fbGetSystemTime(timeLoDW => stFileTime.dwLowDateTime,
                timeHiDW => stFileTime.dwHighDateTime);
astEventBuffer[nCurrentIndex].dtTimestamp := FILETIME_TO_DT(
    stFileTime);

// Update index and count
```

```
nCurrentIndex := nCurrentIndex + 1;
nEventCount   := nEventCount + 1;

// Return success
AddEvent := TRUE;
```

Usage Example

```
// Program: MAIN
// Demonstrating the use of FB_EventLogger
PROGRAM MAIN
VAR
    fbLogger     : FB_EventLogger;     // Instance of the event logger
    bAddResult   : BOOL;               // Result of AddEvent call
    bTrigger     : BOOL;               // Trigger to add an event
END_VAR

// Add an event when triggered
IF bTrigger THEN
    bAddResult := fbLogger.AddEvent(
        nEventId    := 1001,
        sEventText  := 'System started successfully',
        eSseverity  := TcEventSeverity.Info
    );
    bTrigger := FALSE;  // Reset trigger
END_IF
```

Quick Reference Card

Function Block Declaration Syntax

```
FUNCTION_BLOCK <Name>
// Or with inheritance:
FUNCTION_BLOCK <Name> EXTENDS <ParentFB>
VAR_INPUT
    // Input parameters
END_VAR
VAR_OUTPUT
    // Output parameters
END_VAR
VAR
    // Internal variables (state)
END_VAR
VAR CONSTANT
    // Named constants
END_VAR

// FB Body code here (runs every cycle when FB is called)
```

Method Declaration Syntax

```
METHOD <AccessSpecifier> <MethodName> : <ReturnType>
VAR_INPUT
```

```
    // Input parameters
END_VAR
VAR
    // Local variables
END_VAR

// Method implementation
<MethodName> := <returnValue>;
```

OOP Keywords Summary

Keyword	Purpose
`EXTENDS`	Inherit from a parent function block
`IMPLEMENTS`	Implement an interface
`THIS^`	Reference to the current instance
`SUPER^`	Reference to the parent class
`METHOD`	Define a method within an FB
`PROPERTY`	Define a property (getter/setter)
`VAR CONSTANT`	Define named constants

Table 4: OOP Keywords in TwinCAT 3

Common Naming Conventions

Prefix	Type	Example
`FB_`	Function Block	`FB_Motor`, `FB_EventLogger`
`I_`	Interface	`I_Notification`, `I_Loggable`
`fb`	FB Instance	`fbWinch`, `fbLogger`
`UPPER_CASE`	Constants	`MAXIMUM_BUFFER_SIZE`

Table 5: Naming Conventions for OOP in TwinCAT 3

Last Updated: March 14, 2026